

Django Vue Lab

Занятие 2. Django ORM

Черепанов И.Н. Филиппьев В.А.

Модель

Модели отображают информацию о данных, с которыми вы работаете. Они содержат поля и поведение ваших данных. Обычно одна модель представляет одну таблицу в базе данных.

```
from django.db import models

class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

Модель определяет

- Схема БД
- Миграции
- Маппинг данных из БД в python

Под вопросом

- Доменные объекты?
- Бизнес логика?
-

Классы в Python

```
class Person(object):
    atribut = 'foo'
    def method(self, arg1, arg2, kwarg=1):
        return arg1 * arg2 + kwarg
    def __init__(self, *args, **kwargs):
        pass

class Person2(Person):
    atribut = 'bar'

    def method(self, arg1, arg2, kwarg=1):
        res = super(Person2, self).method(arg1, arg2, kwarg=1)
        return res / 2

person = Person()
person.method()
```

Модель определяет

- Тип колонки в базе данных, например: INTEGER, VARCHAR
- Виджет, используемый при создании поля формы, например: `<input type="text">`, `<select>`
- Минимальные правила проверки данных, используемые в интерфейсе администратора и для автоматического создания формы

Настройка полей

- null
- blank
- choices

```
YEAR_IN_SCHOOL_CHOICES = (  
    ('FR', 'Freshman'),  
    ('SO', 'Sophomore'),  
    ('JR', 'Junior'),  
    ('SR', 'Senior'),  
    ('GR', 'Graduate'),  
)
```

- default
- unique
- primary_key

Пример

```
from django.db import models

class Person(models.Model):
    SHIRT_SIZES = (
        ('S', 'Small'),
        ('M', 'Medium'),
        ('L', 'Large'),
    )
    name = models.CharField(max_length=60)
    shirt_size = models.CharField(max_length=1,
    choices=SHIRT_SIZES, blank=True)
```

Читабельное имя поля

`verbose_name` – определяет название поля в формах и панели администратора.

```
poll = models.ForeignKey(  
    Poll,  
    on_delete=models.CASCADE,  
    verbose_name="the related poll",  
)
```

Миграции

Миграции - это способ Django распространять изменения, которые вы вносите в свои модели (добавление поля, удаление модели и т.д.), в схему вашей базы данных.

- **migrate** - применение и отмена миграции.
- **makemigrations** - создание новых миграций на основе изменений.
- **sqlmigrate**- отображает операторы SQL для миграции.
- **showmigrations**- перечислены миграции проекта и их статус.

Связи

Связь многое-к-одному

```
from django.db import models

class Manufacturer(models.Model):
    # ...
    pass

class Car(models.Model):
    manufacturer = models.ForeignKey(Manufacturer,
    on_delete=models.CASCADE)
    # ...
```

Связи

Связь много-ко-многому

```
from django.db import models

class Topping(models.Model):
    # ...
    pass

class Pizza(models.Model):
    # ...
    toppings = models.ManyToManyField(Topping)
```

Связи

Связь один-к-одному

```
from django.db import models

class Topping(models.Model):
    # ...
    pass

class Pizza(models.Model):
    # ...
    toppings = models.OneToOneField(Topping)
```

Мета настройки

```
from django.db import models

class Ox(models.Model):
    horn_length = models.IntegerField()

    class Meta:
        ordering = ["horn_length"]
        verbose_name_plural = "oxen"
```

Методы модели

- `__str__()`
- `__unicode__()`
- `get_absolute_url()`
- `save()`

```
from django.db import models

class Blog(models.Model):
    name = models.CharField(max_length=100)
    tagline = models.TextField()

    def save(self, *args, **kwargs):
        do_something()
        #super(Blog, self).save(*args, **kwargs)
        super().save(*args, **kwargs)
        do_something_else()
```

Наследование моделей

- Абстрактные модели
- Multi-table наследование
- Proxy-модели

Абстрактные модели

```
from django.db import models

class CommonInfo(models.Model):
    name = models.CharField(max_length=100)
    age = models.PositiveIntegerField()

    class Meta:
        abstract = True

class Student(CommonInfo):
    home_group = models.CharField(max_length=5)
```

Multi-table наследование

Это второй тип наследования в Django: каждая модель в иерархии будет независимой. Каждая модель имеет собственную таблицу в базе данных и может быть использована независимо. Наследование использует связь между родительской и дочерней моделью через автоматически созданное поле `OneToOneField`.

```
from django.db import models

class Place(models.Model):
    name = models.CharField(max_length=50)
    address = models.CharField(max_length=80)

class Restaurant(Place):
    serves_hot_dogs = models.BooleanField(default=False)
    serves_pizza = models.BooleanField(default=False)
```


Работа в командной оболочке

```
python manage.py shell
```

- ipython
- Jupyter
- django_extensions

```
python manage.py shell_plus
```

```
import os
import django

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"myproject.settings")
django.setup()
```

Создание объектов

```
from blog.models import Blog

b = Blog(name='Beatles Blog', tagline='All the latest Beatles
news.')
b.save()

b2 = Blog.objects.get(pk=1)
b2.name = 'new name'
b2.save()
```

Сохранение полей

ForeignKey

```
from blog.models import Entry

entry = Entry.objects.get(pk=1)
cheese_blog = Blog.objects.get(name="Cheddar Talk")
entry.blog = cheese_blog
entry.save()
```

ManyToManyField

```
from blog.models import Author

joe = Author.objects.create(name="Joe")
entry.authors.add(joe)
```

Получение объектов

Для получения объектов из базы данных создается QuerySet через Manager модели.

```
all_entries = Entry.objects.all()
```

Фильтры:

- `filter(**kwargs)`
- `exclude(**kwargs)`

```
Entry.objects.filter(  
    headline__startswith='What'  
)  
.exclude(  
    pub_date__gte=datetime.date.today()  
)  
.filter(  
    pub_date__gte=datetime(2005, 1, 30)  
)
```

QuerySet

- Фильтры уникальны
- QuerySet – ленивы

```
q1 = Entry.objects.filter(headline__startswith="What")
q2 = q1.exclude(pub_date__gte=datetime.date.today())
q3 = q1.filter(pub_date__gte=datetime.date.today())

q = Entry.objects.filter(headline__startswith="What")
q = q.filter(pub_date__lte=datetime.date.today())
q = q.exclude(body_text__icontains="food")
print(q)
```

Получение одного объекта с помощью get

ТОЛЬКО ОДИН ОБЪЕКТ

```
one_entry = Entry.objects.get(pk=1)
```

Фильтры полей

Фильтры полей – это операторы для составления условий SQL WHERE. Они задаются как именованные аргументы для метода `filter()`, `exclude()` и `get()` в `QuerySet`.

Фильтры полей выглядят как `field__lookuptype=value`. Используется двойное подчеркивание.

- Связные модели `id` – исключение

```
Entry.objects.filter(blog_id=4)
```

- `exact`

```
Entry.objects.get(headline__exact="Cat bites dog")
```

Фильтры полей

- iexact
- contains
- icontains
- startswith, endswith

Фильтры по связанным объектам

Django предлагает удобный и понятный интерфейс для фильтрации по связанным объектам, самостоятельно заботясь о JOIN в SQL. Для фильтра по полю из связанных моделей используйте имена связывающих полей, разделенных двойным нижним подчеркиванием, пока вы не достигните нужного поля.

```
Entry.objects.filter(blog__name='Beatles Blog')
Blog.objects.filter(entry__headline__contains='Lennon')
Blog.objects.filter(entry__authors__name='Lennon')
Blog.objects.filter(
    entry__authors__isnull=False,
    entry__authors__name__isnull=True,
)
```

Сравнение, копирование, удаление

Сравнение

```
some_entry == other_entry  
some_entry.id == other_entry.id
```

Копирование

```
blog = Blog(name='My blog', tagline='Bloggng is easy')  
blog.save() # blog.pk == 1  
  
blog.pk = None  
blog.save() # blog.pk == 2
```

Удаление

```
e.delete()
```

Связь один-к-многим

Прямая

```
e = Entry.objects.get(id=2)
e.blog # Returns the related Blog object.
```

```
e = Entry.objects.get(id=2)
e.blog = some_blog
e.save()
```

Обратная

```
b = Blog.objects.get(id=1)
b.entry_set.all()
```

Спасибо за внимание

Навык “Web разработка” повышается